Computer Architecture Lab

(EL-2012)

LABORATORY REPORT

Fall 2026



(Lab#08)

Implementation of Program Counter and Instruction Memory

Student Name: _____*Syed Sikandar Abbas*___ Roll No: __*i246553*___ Section: *CE-B*

Date performed: ____*October 14*, 2026

MARKS AWARDED: _____ / 10

Signature and date of Instructor: _____
Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad

Task 1:

Description:

Implemented a program counter to support a 4-bit address. Simulated and verified the results using the same testbench.

Code:

```
module program_counter_4bit (
    input clk,
    input reset,
    input [3:0] addr,
    output reg [3:0] pc_out
);

always @(posedge clk or posedge reset) begin
    if (reset)
        pc_out <= 4'b0000;
    else
        pc_out <= addr;
end

endmodule

module tb_program_counter_4bit;
    reg clk;
    reg reset;
    reg [3:0] addr;
    wire [3:0] pc_out;

    program_counter_4bit uut (
        .clk(clk),
        .reset(reset),
        .addr(addr),
        .pc_out(pc_out)
    );

    always begin
        #5 clk = ~clk;
    end

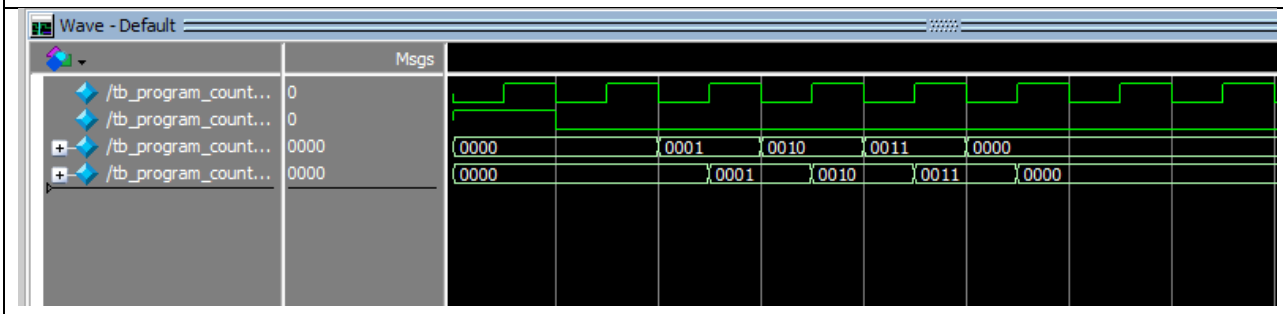
    initial begin
        clk = 0;
        reset = 0;
        addr = 4'b0000;

        reset = 1;
        #10 reset = 0;

        #10 addr = 4'd1;
        #10 addr = 4'd2;
```

```
        #10 addr = 4'd3;
        #10 addr = 4'd0;
    end
endmodule
```

Output:



Task 2:

Description:

Simulated the Program Counter increments automatically by 1 after every clock cycle (instead of setting the address manually). Verified that the instructions are fetched sequentially from the instruction memory. Added error handling in the Instruction Memory module to return a default instruction (e.g., 8'b00000000) if an invalid address is provided. Add error handling in the Instruction Memory module to return a default instruction (e.g., 8'b00000000) if an invalid address is provided.

Code:

```
module pc_auto_increment (
    input clk,
    input reset,
    output reg [7:0] pc_out
);
    always @(posedge clk or posedge reset) begin
        if (reset)
            pc_out <= 8'h00;
        else
            pc_out <= pc_out + 1'b1;
    end
endmodule

module instruction_memory_safe (
    input [7:0] addr,
    output reg [7:0] instruction
);
    reg [7:0] memory [0:15];

    initial begin
        memory[0] = 8'b00000001;
        memory[1] = 8'b00000010;
        memory[2] = 8'b00000011;
        memory[3] = 8'b00000100;
    end

    always @(*) begin
        if (addr > 8'd15)
            instruction = 8'b00000000;
        else
            instruction = memory[addr];
    end
endmodule

module top_task2 (
    input clk,
    input reset,
    output [7:0] pc_out,
    output [7:0] instruction
);
    pc_auto_increment pc_inst (
```

```
        .clk(clk),
        .reset(reset),
        .pc_out(pc_out)
    );

    instruction_memory_safe im_inst (
        .addr(pc_out),
        .instruction(instruction)
    );
endmodule

module tb_top_task2;
    reg clk;
    reg reset;
    wire [7:0] pc_out;
    wire [7:0] instruction;

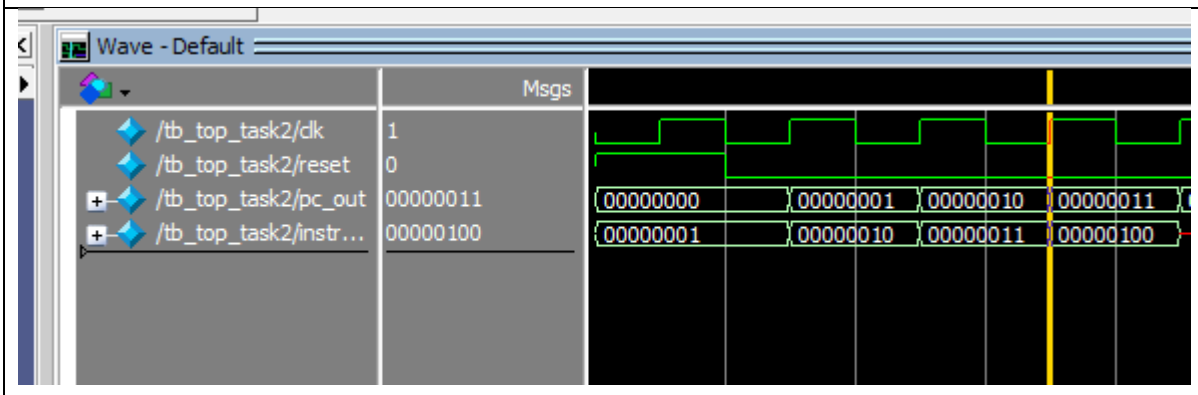
    top_task2 uut (
        .clk(clk),
        .reset(reset),
        .pc_out(pc_out),
        .instruction(instruction)
    );

    always #5 clk = ~clk;

    initial begin
        clk = 0;
        reset = 1;
        #10;
        reset = 0;

        #100;
    end
endmodule
```

Output:



Task 3:

Description:

Implemented a 32-bit PC register compliant with the RV32 architecture. To make simulation manageable within the test bench environment, configured the PC's sequential increment logic to add 4 bytes (+4) on every rising clock edge, ensuring proper 32-bit word alignment. Truncate the lower 4 bits of the PC output to drive a 4-bit address space for the immediate test environment. Simulated the design using the provided testbench. Verified that the PC resets correctly to and holds its value stably between clock edges.

Code:

```
module pc_rv32_4bit (
    input clk,
    input reset,
    output reg [31:0] pc_32bit_out,
    output [3:0] pc_4bit_addr
);

    always @(posedge clk or posedge reset) begin
        if (reset)
            pc_32bit_out <= 32'h00000000;
        else
            pc_32bit_out <= pc_32bit_out + 32'd4;
    end

    assign pc_4bit_addr = pc_32bit_out[5:2];

endmodule

module tb_pc_rv32_4bit;
    reg clk;
    reg reset;
    wire [31:0] pc_32bit_out;
    wire [3:0] pc_4bit_addr;

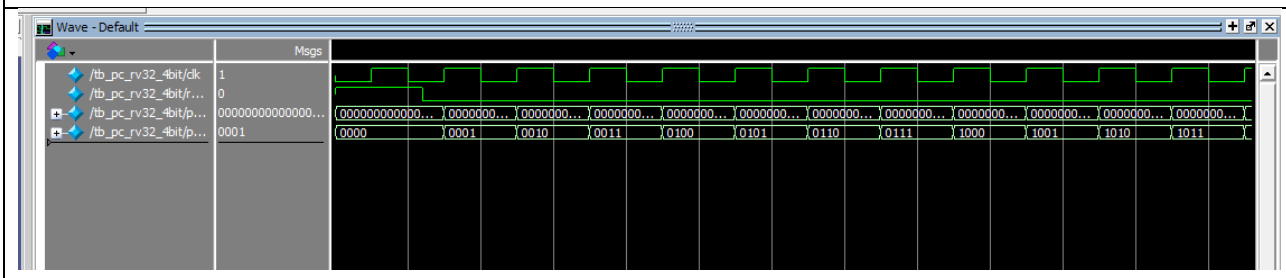
    pc_rv32_4bit uut (
        .clk(clk),
        .reset(reset),
        .pc_32bit_out(pc_32bit_out),
        .pc_4bit_addr(pc_4bit_addr)
    );

    always #5 clk = ~clk;

    initial begin
        clk = 0;
        reset = 1;
        #12;
        reset = 0;
    end
```

```
endmodule
```

Output:



Task 4:

Description:

Integrated the 32-bit PC with a 32-bit wide Instruction Memory (IM) module. The circuit automatically increments the PC by 4 after every clock cycle (simulating sequential execution without manual address intervention). Loaded a sample program into the Instruction Memory. Ran the clock continuously and verified via the test bench waveforms that 32-bit instructions are fetched sequentially from memory addresses (0x00, 0x04, 0x08, 0x0C, etc.) matching the expected program flow.

Code:

```
module rv32_pc_im_integrated (
    input clk,
    input reset,
    output [31:0] pc_out,
    output [31:0] instruction
);
    always @(posedge clk or posedge reset) begin
        if (reset)
            pc_out <= 32'h00000000;
        else
            pc_out <= pc_out + 32'd4;
    end

    reg [31:0] imem [0:15];

    initial begin
        imem[0] = 32'h00000013;
        imem[1] = 32'h00100093;
        imem[2] = 32'h00200113;
        imem[3] = 00300193;
    end


    assign instruction = imem[pc_out[5:2]];

endmodule

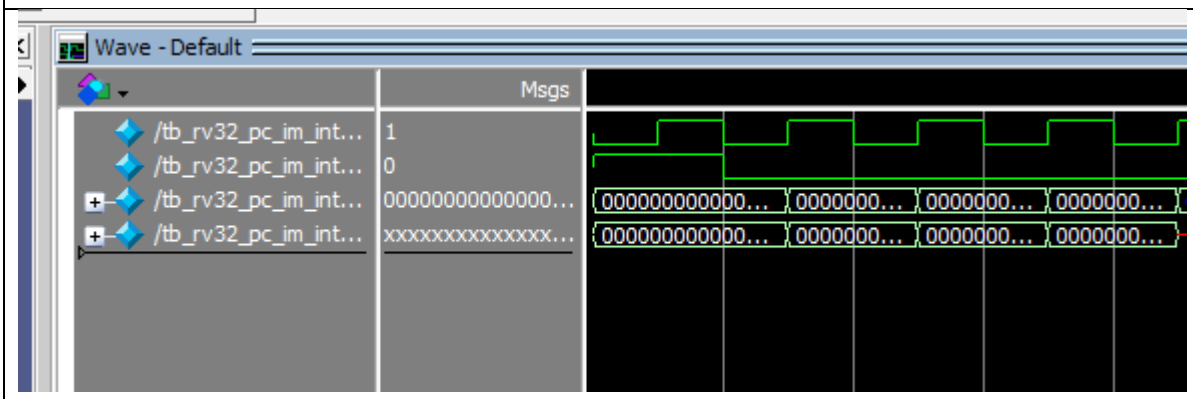
module tb_rv32_pc_im_integrated;
    reg clk;
    reg reset;
    wire [31:0] pc_out;
    wire [31:0] instruction;

    rv32_pc_im_integrated uut (
        .clk(clk),
        .reset(reset),
        .pc_out(pc_out),
        .instruction(instruction)
    );

    always #5 clk = ~clk;
```

```
    initial begin
        clk = 0;
        reset = 1;
        #10;
        reset = 0;
    end
endmodule
```

Output:



Task 5:

Description:
32-bit RISC-V PC and an IM module using Logisim.
Circuit:

